
Table of Contents

.....	1
Lab 6, Images in Greyscale and RGB.	1
Exercise 6.1	1
Exercise 6.2	2
Functions	3

```
clear all; close all; clc;
```

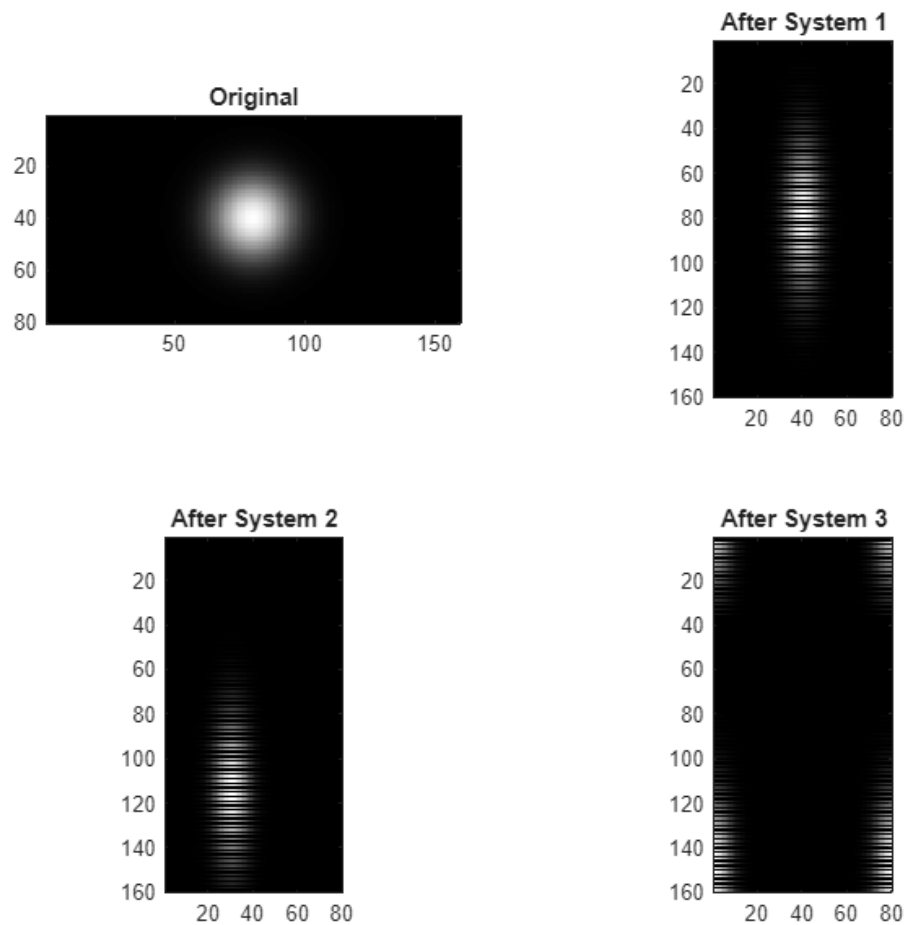
Lab 6, Images in Greyscale and RGB.

Exercise 6.1

```
% USER DEFINED VARIABLES
w = 15; % Width
x = 1:160; % Horiztonal Axis
y = 1:80; % Vertical Axis
% == > Our grayscale value, x and y define location, z the value, centered
% (80, 40) <==
z = round (127* exp ( -1/ w .^2*(( y .' -40) .^2+( x -80) .^2) ) ) ;
% == > Lets MATLAB know we are working in grayscale <==
colormap ( gray ) ;
% == > Downsamples half of our x values, and upsamples our y axis <==
[ xs , ys , zs ] = image_system1 ( z ,2 ,2) ;
za = image_system2 ( zs , -10 ,35) ;
zb = image_system3 ( za , -30 ,35) ;

% PLOT RESULT WITH SUBPLOT
figure (1) ;
subplot (2 ,2 ,1) ; % == > Create 2x2 plot grid, place in top left <==
imagesc ( x , y , z ) ; % == > Display image with x y position and z value <==
axis image ; % == > 1:1 aspect ratio for image to display clearly <==
title ( ' Original ' )
subplot (2 ,2 ,2) ; % == > Places in top right of 2x2 grid <==
imagesc ( xs , ys , zs ) ; % == > Display image with xs ys position and zs
value <==
axis image ; % == > 1:1 aspect ratio for image to display clearly <==
title ( ' After System 1 ' )

subplot (2 ,2 ,3) ; % == > Places in bottom left of 2x2 grid <==
imagesc ( xs , ys , za ) ; % == > Display image in xs ys position and za
value <==
axis image ; % == > 1:1 aspect ratio for image to display clearly <==
title ( ' After System 2 ' )
subplot (2 ,2 ,4) ; % == > Places in bottom right of 2x2 grid <==
imagesc ( xs , ys , zb ) ; % == > Display image in xs ys position and zb
value <==
axis image ; % == > 1:1 aspect ratio for image to display clearly <==
title ( ' After System 3 ' )
```



Exercise 6.2

```
z2 = load('lighthouse.mat');  
data = z2.xx;  
  
[xs, ys, zf] = image_sample(data, 2);  
  
figure(2);  
imagesc(xs, ys, zf);  
colormap(gray);  
axis image;  
title('Lighthouse');
```



Functions

```
function [ xs , ys , zs ] = image_system1 ( z , Dx , Uy )

    % IMAGE_SYSTEM1 === > Takes original z matrix and upsamples/downsamples
<===
    % == > Pre-allocates memory to the new down/upsampled matrix <==
    zs = zeros ( ceil ( Uy * size ( z ,1) ) , ceil ( size ( z ,2) / Dx ) ) ;
    % == > Uy upsamples by taking Uy times as many row values and Dx
downsamples
    % by dividing the amount of samples by Dx <==
    ys = 1: ceil ( Uy * size ( z ,1) ) ;
    xs = 1: ceil ( size ( z ,2) / Dx ) ;
    % == > Takes every Uy rows and takes a column every Dx columns <==
    zs (1: Uy : end ,1: end ) = z (1: end ,1: Dx : end ) ;

end

function [ za ] = image_system2 ( z , Sx , Sy )
```

```

    % IMAGE_SYSTEM2 === > Transforms the image, shifting it based on Sx, Sy
<===
    % ==== > Creates a 2d matrix with the same rows and column data as z
<====
    za = zeros ( size (z ,1) , size (z ,2) ) ;
    for nn = 1: size (z ,1)
        for mm = 1: size (z ,2)
            % ==== > Checks if nn - Sy and mm - Sx are within the image bounds
when transformed <====
                if nn > Sy && nn - Sy < size (z ,1) && mm > Sx && mm - Sx < size
(z ,2)
                    % ==== > Lowers z value, dimming, and transforms the image
<====
                        za ( nn , mm ) = 1/2* z ( nn - Sy , mm - Sx ) ;
                    end
                end
            end
        end
    end

function [ zb ] = image_system3 ( za , Sx , Sy )

    % IMAGE_SYSTEM3 === > Causes the image to invert, wrapping it around the
edge of the frame <===

    % ==== > Recreates the matrix with one row and column removed <====
    x = 0:1: size ( za ,2) -1;
    y = 0:1: size ( za ,1) -1;

    % ==== > Uses modulus to shift and wrap around our rows and columns <====
    xs = mod (x - Sx , size ( za ,2) ) ;
    ys = mod (y - Sy , size ( za ,1) ) ;

    % ==== > Adds one so we dont have zb(0) and recreates matrix with
wraparound <====
    zb = za ( ys +1 , xs +1) ;

end

function [xs, ys, zs] = image_sample(z, D)

    ys = 1:D:size(z,1);
    xs = 1:D:size(z,2);
    zs = z(ys, xs);

end

function [zaa, x, y] = image_antialias(z, x2, y2)

end

```